Développement Web PHP

BUT 2
Pascal NITSCHKE - Zsuzsanna ROKA

pascal.nitschke@univ-lorraine.fr - roka5@univ-lorraine.fr

Séances

32h

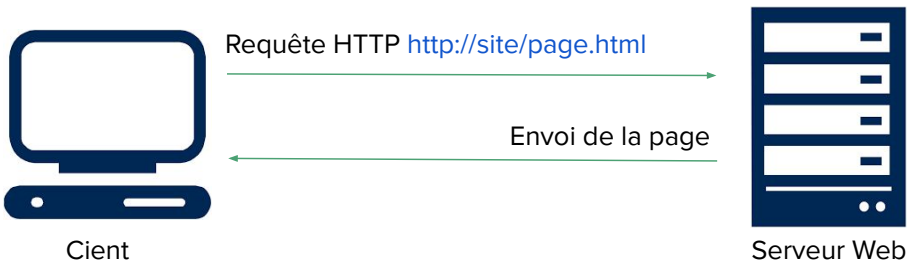
- Introduction et notions de base
- Classes
- Formulaires
- Contrôle (coef 1)
- Fichiers + JSON
- Base de données + sessions
- TP (noté, coef 3)

1. Introduction et notions de base

Nitschke - Roka

BUT 1 DevWeb - Rappel

- HTML5 (contenu)
- CSS 3 (style)
- Sites statiques
- Besoins : éditeur de texte + navigateur



BUT 2 DevWeb

- Sites dynamiques (connexion, formulaire, forum, e-commerce, ...)
- Besoins :
  - Serveur Web (Apache, ...)
  - Base de données (MySQL, MariaDB, ...)
  - PHP (PHP 8)

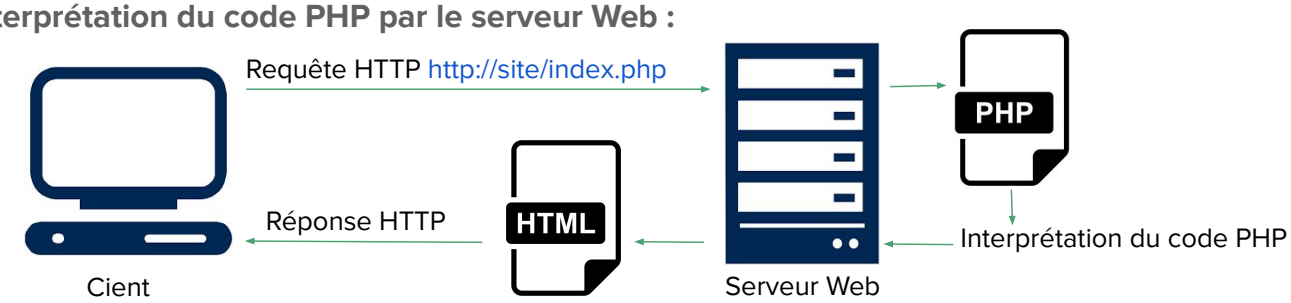
Le langage PHP

Personal Home Page (historiquement)

Hypertext PreProcessor (officiellement)

- Langage de scripts, exécuté côté serveur
- Code inclus dans une page HTML
- Orienté objet
- Couplé avec HTML, CSS et MariaDB
- Permet de réaliser des pages Web dynamiques
- Documentation : http://www.php.net/manual/fr/langref.php

Interprétation du code PHP par le serveur Web :




```
<!DOCTYPE html>
<html>
  <head>
      <title>Exemple</title>
  </head>
  <body>
      <p>Hello world !</p>
  </body>
</html>
```

```
<!DOCTYPE html>
<html>
  <head>
      <title>Exemple</title>
  </head>
  <body>
      <?php
          echo '<p>Hello world !</p>' ;
      ?>
  </body>
</html>
```


Historique du langage PHP

1994 : Rasmus Lerdorf (Groenland)
1995 : PHP 2
1998 : PHP 3
2000 : PHP 4 (moteur d'analyse Zend)
2004 : PHP 5 (développement orienté objet)
2015 : PHP 7 (pas de PHP 6)
2020 : **PHP 8** (gestion des types, des erreurs)

Structure de base

- Nom du fichier : nomfichier**.php**
- Code inclus dans les balises PHP
  - **<?php**
    code
    ?>
  - <?=
    Code
    ?> seulement si *short_open_tag = on* dans *php.ini*

    <?php echo $variable ?> peut être remplacé par <?= $variable ?>
- Séparateur d'instruction : le caractère ";"

Structure de base - suite

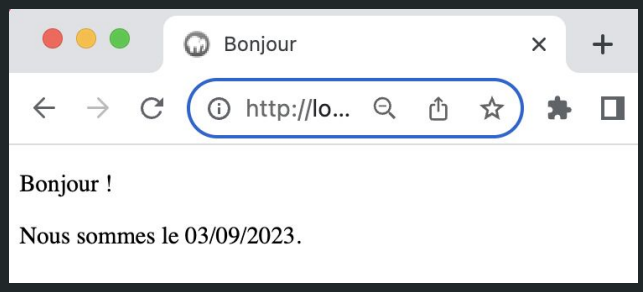
- Commentaires :
  - Sur une ligne
    // ceci est un commentaire ou
    # ceci est un commentaire
  - Sur plusieurs lignes
    /*
    Ceci est un commentaire
    */
  - Pour une function/méthode (comme en Java)
    /**
    * Ceci est un commentaire
    */
- Règles de nommage : nomVariable, nomFonction, NomClasse, nomtype

Exemple

```
<?php
        $aujourdhui = date("d/m/Y");
?>
<!DOCTYPE html>
<html>
  <head>
     <title>Bonjour</title>
     <meta charset = “UTF-8” />
  </head>
  <body>
    <?php
      // on affiche la date d'aujourd'hui
      echo "<p>Bonjour !</p>";
      echo "<p>Nous sommes le $aujourdhui.</p>";
    ?>
  </body>
</html>
```

“Afficher le code source de la page” :

```
<!DOCTYPE html>
<html>
  <head>
      <title>Bonjour</title>
      <meta charset = “UTF-8” />
  </head>
  <body>
      <p>Bonjour !</p>
      <p>Nous sommes le 03/09/2023.</p>
  </body>
</html>
```



Exemple

```
<?php
        $aujourdhui = date("d/m/Y");
?>
<!DOCTYPE html>
<html>
  <head>
     <title>Bonjour</title>
     <meta charset = “UTF-8” />
  </head>
  <body>
    <?php
      // on affiche la date d'aujourd'hui
      echo "<p>Bonjour !</p>";
      echo "<p>Nous sommes le $aujourdhui.</p>";
    ?>
  </body>
</html>
```

Le même rendu

```
<body>
  <p>Bonjour !</p>
  <?php
      echo "<p>Nous sommes le $aujourdhui.</p>";
  ?>
</body>
```

```
<body>
  <p>Bonjour !</p>
  <p>Nous sommes le  <?php echo $aujourdhui; ?>.</p>
</body>
```

```
<body>
  <p>Bonjour !</p>
  <p>Nous sommes le  <?= $aujourdhui ?>.</p>
</body>
```

Les bases du langage

Constantes

Variables

Types des données

Opérateurs

Structures de contrôle (conditionnel, boucles)

Tableaux

Fonctions

Constantes

- define ('PI', 3.14);
  // nom entre apostrophes
- const UNE_CHAINE = "valeur de PI";
  // nom sans apostrophes
- Sensible à la casse
- Portée : le script dans lequel elle est définie

```
echo PI, " : ", UNE_CHAINE;
echo pi;
```

3.14 : valeur de PI
Fatal error: Uncaught Error: Undefined constant "pi" in...

Variables

- Nom des variables :
  - préfixé par $
  - commence par une lettre
  - lettres, chiffres, “_”
- Définies lors de leur première utilisation
  - pas de déclaration explicite, 0 par défaut
- Typées automatiquement, peuvent être redéfinies
- Sensibles à la casse
- Affectation d’une valeur : avec l’opérateur " = "
- Portée et durée de vie : le script dans lequel elle est définie ; le temps de l’exécution du script

```
$nom = "Dupont";
echo $nom, "<br />" ;
$nom = 3.14;
echo $nom, "<br />" ;
echo $noM;


Dupont
3.14
Warning: Undefined variable $noM in
...
```


Variables

Afficher la valeur d’une variable

type scalaire : echo

```
echo $prenom, " ", $nom;
echo $prenom." ".$nom;
echo($prenom." ".$nom); (une seule valeur !)
```

Affichage à des fins de test (tout type) :

- var_dump($var);
- print_r($var);
- var_export($var);

```
$var = ['zero' => 0, 'un' => 1];

var_dump($var) :
   array(2) { ["zero"]=> int(0) ["un"]=> int(1) }

print_r($var) :
   Array ( [zero] => 0 [un] => 1 )

var_export($var);
   array ( 'zero' => 0, 'un' => 1, )
```

Types des données

- Scalaires
  - Entier — int
  - Nombre à virgule flottant — float, double
  - Chaîne de caractères — string
  - Booléen — bool
- Types composés
  - Tableau — array
  - Objet — object
- Types spéciaux
  - NULL — null
  - Ressource — resource
  - Fonction de rappel — callable
  - Itérable — iterable

```
$entier = 123456;                    123456
$autreEntier = 123_456;              123456
$x = 1.5;                            1.5
$y = 1.5E2                           150
$chaine = "C'est une phrase".
$chaine = 'C\'est une autre phrase'.
```

Conversion chaîne => nombre :

Seulement si la chaîne correspond à un nombre.

Exemples :

```
"1.5E2" + 1 = 151
1 + "1xxx" = 2        + Warning
1 + "x11"             Fatal error
```


Fonctions PHP - constantes, variables

Constantes

fonction(CONSTANTE)

defined (définie ou non)
constant (valeur)

Variables

fonction(variable)

empty (vide ou non)
isset (définie ou non)
unset (suppression)
var_dump (type et valeur)

Conversions

(type)valeur

(int) ou (integer)
(bool) ou (boolean)
(double) ou (float)
(string)
(array)
(object)

```
                    var_dump :
isset($w)           = bool(false)
$w = 0;
$w                  = int(0)
isset($w)           = bool(true)
empty($w)           = bool(true)
```

```
                    var_dump :
$w = "Dupont";
$w                  = string(6) "Dupont"
empty($w)           = bool(false)
unset($w);
isset($w)           = bool(false)
```

```
                    var_dump :
(int)1.25           = int(1)
(string)1.25        = string(4) "1.25"
$x="123a";
(float)$x;          = float(123)
```

Fonctions PHP - conversions

Fonction de conversion (explicite)

settype (variable, 'type');
code plus lisible...

```
                          var_dump :
$x = "123a";
settype($x, 'integer');   = int(123)
settype($x, 'string');    = string(3) "123"
```

Test du type d'une variable

is_array, is_bool, is_int, is_float (is_double), is_null, is_countable, is_numeric, is_string, is_object, ...

```
                          var_dump :
$x = "123a";
is_string($x)             = bool(true)
is_numeric($x)            = bool(false)
```

Autres fonctions de conversion

intval, floatval, strval, boolval

```
                          var_dump :
$x = 123a;
strval($x)                = string(4) "123a"
(string)$x                = string(4) "123a"
```


Fonctions PHP - chaînes de caractères

Valeur entre " " ou ' ' (pas de type 'character')

Quelques fonctions pratiques :

Concatenation : . (le point)

Longueur de la chaîne : strlen

Conversion min/maj : strtolower, strtoupper, ucfirst, ucwords, lcfirst

Comparaison : strcmp, strcasecmp

Suppression des blancs en début et en fin de chaîne : ltrim (début), rtrim (fin), trim (début et fin)

```
$chaine = "  Ceci  est une phrase avec plein
d'espaces.  ";

strlen($chaine)                 = 46
strlen(trim($chaine))           = 42

strcmp('plein', 'PLEIN')        int(32)
strcasecmp('plein', 'PLEIN')    int(0)

$chaine = strtolower('Bonjour MR PATATE !');

$chaine                 = bonjour mr patate !
ucfirst($chaine)        = Bonjour mr patate !
ucwords($chaine)        = Bonjour Mr Patate !
```

Fonctions PHP - chaînes de caractères

" " ou ' ' ? (pas de type ‘character’)

Différence :
" " permet d’afficher la valeur d’une variable

```
<?php echo "La valeur de 'v' est $v"; ?>
<?php echo 'La valeur de \'v\' est $v'; ?>
<?php echo 'La valeur de \'v\' est ', $v; ?>
La valeur de 'v' est <?= $v?>  (short tag)
```

PHP 8:

str_contains *(sensible à la casse)*
str_starts_with
str_ends_with

```
$v = 5;

La valeur de 'v' est 5
La valeur de 'v' est $v
La valeur de 'v' est 5
La valeur de 'v' est 5

str_contains($chaine, 'PLEIN')
            = bool(false)
str_contains(strtoupper($chaine), 'PLEIN')
            = bool(true)
```


Opérateurs basiques

- Affectation par valeur : =
  $x = 0; $a = $b = $c = 1.5; (affectation multiple)
- Arithmétiques : +, -, *, /, **, %, ++, - -
- Concaténation des chaînes : . (le point)
  "Pierre"." Dupont" : "Pierre Dupont"
- Comparaison : ==, ===, !=, !==, <, >, <=, >=
  $x = 0; $y = "0";
  $x == $y : TRUE mais $x === $y : FALSE
  "===" : les types doivent être également identiques

  Les nombres sont d’abord transformés en chaînes pour être comparés (PHP 8)
- Logiques : and, &&, or, ||, xor, !

Structures de contrôle - conditionnelles

La structure if

```
if (condition1) {
        instructions1;
}
elseif (condition2) {
        instructions2;
}
...
else {
        instructionsN;
}
```

Syntaxe alternative

```
if (condition1) :
        instructions1;
elseif (condition2) :
        instructions2;
...
else :
        instructionsN;
endif;
```

Pratique si les instructions sont des codes HTML

La structure switch

```
switch (expression) {
  case valeur1 :
        instructions1;
        break;
  case valeur2 :
        instructions2;
        break;
  ...
  default :
        instructionsN;
}
```

Syntaxe alternative

```
switch (expression) :
  case expression1 :
        instructions1;
        break;
  case expression2 :
        instructions2;
        break;
  ...
  default :
        instructionsN;
endswitch;
```


Structures de contrôle - boucles

La boucle while

```
while (condition) {
        instructions;
}
```

Syntaxe alternative

```
while (condition) :
        instructions;
endwhile;
```

La boucle for

```
for (exp1; exp2; exp3) {
        instructions;
}
```

Syntaxe alternative

```
for (exp1; exp2; exp3) :
        instructions;
endfor;
```

La boucle do ... while

```
do {
        instructions;
} while (condition);
```

```
<?php
    for ($i=1; $i<=10; $i++){
        echo $i, " ";
    }
?>

<?php for ($i=1; $i<=5; $i++): ?>
    <p>Pierre Dupont </p> <!-- html -->
<?php endfor; ?>
```

Tableaux - numérique et associatif

Tableau numérique

$tab1

Indice/Clé	Valeur
0	pomme
1	abricot
2	banane
3	orange

$tab2

Indice/Clé	Valeur
30	pomme
3	abricot
1	banane
15	orange

Tableau associatif

$tab3

Indice/Clé	Valeur
zero	pomme
1	abricot
deux	banane
15	orange

Accès à une case :

$tab1 [2] = "banane"

$tab2 [15]= "orange"

$tab3 [**'zero'**]= "pomme"

$tab3 [zero] **Fatal error !**

Tableaux - création (classique)

- Sans “array” :

```
$tab []        = 1
$tab []        = 3
$tab [5]       = 5
$tab []        = 6
$tab ['sept'] = 7
```

Indice/Clé	Valeur
0	1
1	3
5	5
6	6
sept	7

indice max (=5) + 1 = 6

- Avec “array” :

```
$entiers = array (1, 3, 5, 7);  // indices : 0, 1, 2, 3

$entiers = array(); // tableau vide

$personne = array(      'nom' => "Dupont”,
                        'prenom' => "Pierre",
                        'age' => 18)

$tab2d = array(   array ('orange', 'banane', 'pomme'),
                  array('courgette', tomate'));
```

Tableaux - syntaxe courte

➢ Syntaxe courte :

```
$entiers =      [1, 3, 5, 7];
$tab =          ['zéro', 'un', 3 => 'trois', 'quatre' => 4];
$personne =     [  'nom' => "Dupont”,
                   'prenom' => "Pierre",
                   'age' => 18]

$tab2d = [      ['orange', 'banane', 'pomme'],
                ['courgette', poivron]];

$tab2dass =     [  'fruits' =>      ['orange', ...],
                   'legumes' => ['courgette', ...]];
```

$entiers

0	1	2	3
1	3	5	7

$tab

0	1	3	'quatre'
'zéro'	'un'	'trois'	4

$personne

'nom'	'prenom'	'age'
'Dupont'	'Pierre'	18

$tab2d

0			1	
0	1	2	0	1
'orange'	'banane'	'pomme'	'courgette'	'poivron'

$tab2dass

'fruits'			'légumes'	
0	1	2	0	1
'orange'	'banane'	'pomme'	'courgette'	'poivron'

Tableaux - parcours

➔ Boucle for
➔ Boucle while
➔ **Boucle foreach**

➢ Affichage des valeurs uniquement :

```
foreach ($tableau as $uneValeur){
    echo $uneValeur, " ";
}
```

➢ Affichage des clés/indices et des valeurs :

```
foreach ($tableau as $cle => $uneValeur){
    echo $cle, " => ", $uneValeur, "<br /> ";
}
```

```
$tableau = array ('zéro',
                  'un',
                  3 => 'trois',
                  'quatre' => 4);
```

Valeurs :
zéro un trois 4

Clés + valeurs :
0 => zéro
1 => un
3 => trois
quatre => 4

Fonctions PHP - tableaux

count : nombre d'éléments dans le tableau
(array_push : ajoute en fin de tableau)
array_pop : retire le dernier élément
array_unshift : ajoute en début de tableau
array_shift : retire le premier élément
explode, implode : chaîne ⇔ tableau

in_array, search_array, array_replace
sort, asort, ksort, rsort (tri)
max, min
strsplit
array_column (pour tableaux à dim > 1)
array_keys, array_key_first, array_key_last,
array_key_exists

```
$tab = [1, 2, 'trois', 'quatre' => 4];
                                var_dump :
in_array(4, $tab);              = bool(true)
in_array('4', $tab);            = bool(true)
in_array('4', $tab, TRUE);      = bool(false)
array_search(4, $tab)           = "quatre" (son indice)
array_search('cinq', $tab)      = bool(false)

$chaine = "a,b,c";
explode(',', $chaine)           array(4) {
                                      [0]=> string(1) "a"
                                      [1]=>  string(1) "b"
                                      [2]=> string(1) "c"
                                }
```


Écrire des fonctions

Définition d'une fonction

function nomFonction ([paramètre] [:type]){
    instructions;
}

```
bonjour();                          Bonjour
echo addition(1, 2, 3);             6
echo addition(1, ...[4, 5]);        10

echo nom("DUPONT", "Pierre")[0];  DUPONT

Notation "...[tableau]" :
    ...[1,2,3,4] => 1,2,3,4 (liste de paramètres)
```

```
// fonction sans valeur retournée
// = "procédure"
function bonjour() {
    echo "Bonjour";
}

// fonction avec valeur retournée
function addition($a, $b, $c) {
    return ($a + $b + $c);
}

// fonction retournant un tableau
function nom($nom, $prenom) {
    return ( [$nom, $prenom] );
}
```

Écrire des fonctions - avec type de retour

Définition avec type de retour

function nomFonction ([paramètre] [:type]) : type {
    Instructions;
}

```
bonjour2();                    Bonjour

echo addition(1.5, 2, 3);      6.5
echo addition2(1.5, 2, 3);     6
echo addition3("1.5", 2, 3);   6.5
```

```
// fonction sans valeur retournée
function bonjour2() : void {
        echo "Bonjour";
}

// fonction avec valeur retournée
function addition2($a, $b, $c) : int {
    return ($a + $b + $c);
}

// plusieurs types de retour
function addition3($a, $b, $c) : int | float {
        return ($a + $b + $c);
}
```


Écrire des fonctions - paramètres

Paramètres

type nomParamètre [= valeur]

Forcer le typage (partout dans le script) :

declare(strict_types=1);

```
echo addition("1.5", 2);          3.5
echo addition("1.5", "2");        3.5
echo addition(2, 1.5);            3

echo addition2(1.5, 2);           13.5
echo addition2(1.5, 2, 1);        4.5

echo addition3(2.5, 1)            3.5
echo addition3(1, 2.5)
Fatal error: Uncaught TypeError: addition3(): Argument
#2 ($b) must be of type int
```

```
function addition($a, int $b) : int | float {
    return ($a + $b);
}

//avec une valeur par défaut (en fin de liste)
function addition2($a, int $b, $c=10) : int | float {
    return ($a + $b + $c);
}

// en première ligne du script
declare(strict_types=1);

function addition3($a, int $b) : int | float {
        return ($a + $b);
}
```

Écrire des fonctions - variables

Variables locales/globales

- ➢ définies dans les fonctions : locales
  - non-définies à l'extérieur de la fonction
  - initialisées à chaque appel
- ➢ définies à l'extérieur des fonctions
  - non-définies à l'intérieur de la fonction

Variables accessibles partout :

- global variable;
- utilisation de la variable $GLOBALS (tableau associatif, accessible partout)

```
function variables($x, $y){
        $v = $x +$y;
        return $v;
}
echo $v;                    non-définie
variables(2,3);             5
echo $v;                    non-définie

$v = 10;
function variables($x, $y){
        global $v;
        $v = $x +$y;
        return $v;
}
echo $v;                    non-définie
echo $v;                    10
echo variables(2,3);        5
echo $v;                    5
```


Écrire des fonctions - variables statiques

Variables statiques

- ➢ Conservent leur valeur pendant la durée du script
  - static $variable = valeur initiale;

```
function vStatique($x, $y){
        static $compteur = 1;
        echo "appel numéro ", $compteur, " : ";
        $v = $x +$y;
        $compteur++;
        return $v;
}

echo vStatique(2,3);    appel numéro 1 : 5
echo vStatique(4,3);    appel numéro 2 : 7
echo vStatique(10,7);   appel numéro 3 : 17
```

Inclure un fichier externe

- include 'fichier.php';
  si fichier non existant : warning
- require 'fichier.inc';
  si fichier non existant : erreur et interruption du script

Inclusion seulement une fois :

- include_once 'fichier.php';
- require_once 'fichier.inc';

Pour aller plus loin…

L'opérateur combiné : +=, .=, etc.

L'opérateur ternaire : expression1 ? expression2 : expression3

```
$sexe = 'F';
echo $sexe=='F' ? 'Madame' : 'Monsieur' ;
```

L'opérateur de comparaison combinée : expression1 <=> expression2

-1 si expression1 < expression2, 0 si égales, 1 si expression1 > expression2

L'expression match (PHP 8) : comme switch mais retourne une valeur

```
$a = 2; $b = 3; $op = '+';
$resultat =
    match ($op){
        '+' => $a + $b,
        '-' => $a - $b,
        default => 0
    }
```

Manipulation des dates : checkdate, date, date_parse_from_format, hrtime, etc.
Expressions régulières
Fonctions anonyme, fléchée, générateur